

RECURRENCE RELATION

- A **recurrence relation** is a mathematical equation that describes a sequence of values, where each term in the sequence is defined in terms of previous terms.
- In the context of Design and Analysis of Algorithms (DAA), recurrence relations are often used to analyze the time complexity of algorithms by expressing the time complexity of a problem of **size n** in terms of the time complexity of smaller instances of the same problem.

The general form of a recurrence relation is often expressed as:

$$T(n) = f(n) + T(g(n))$$

where:

- **T(n)** is the time complexity of the problem of **size**.
- **f(n)** is the time complexity of the basic operations at the **current level**,
- **g(n)** represents the **size** of subproblems at the **next level**.

There are **Four** methods commonly used to solve recurrence relations:

- ✚ **Substitution Method**
- ✚ **Recursion Tree**
- ✚ **Iterations Method**
- ✚ **Master Theorem**

❖ Substitution Method

- Guess the form of the solution.
- Use mathematical induction to prove that the guess is correct.

For example consider the recurrence

$$T(n) = T\left(\frac{n}{2}\right) + n$$

We have to show that it is asymptotically bound by **$O(\log n)$** .

Solution:

For **$T(n) = O(\log n)$**

We have to show that for some constant **c**

$T(n) \leq c \log n$.

Put this in given Recurrence Equation.

Now,

$$\begin{aligned} T(n) &\leq c \log \left(\frac{n}{2}\right) + 1 \\ &\leq c \log \left(\frac{n}{2}\right) + 1 = c \log n - c \log_2 2 + 1 \\ &\leq c \log n \text{ for } c \geq 1 \end{aligned}$$

Thus **$T(n) = O(\log n)$** .

Example2. Consider the Recurrence

$$T(n) = 2T\left(\frac{n}{2}\right) + n \quad n > 1$$

Find an Asymptotic bound on **T** .

Solution:

We guess the solution is **$O(n \log n)$** . Thus for constant ' **c** '.

For **$T(n) \leq c n \log n$**

Put this in given Recurrence Equation.

Now,

$$\begin{aligned} T(n) &\leq 2c \left(\frac{n}{2}\right) \log \left(\frac{n}{2}\right) + n \\ &\leq c n \log n - c n \log_2 2 + n \end{aligned}$$

$$=cn \log n - n(c \log 2 - 1)$$

$$\leq cn \log n \text{ for } (c \geq 1)$$

Thus **$T(n) = O(n \log n)$**

The time Complexity of **Substitution Method** is **$O(n \log n)$** , and the space complexity is often the same unless additional space complexities (like recursion stack) need to be considered.

©Topperworld

❖ Recursion Tree Method:

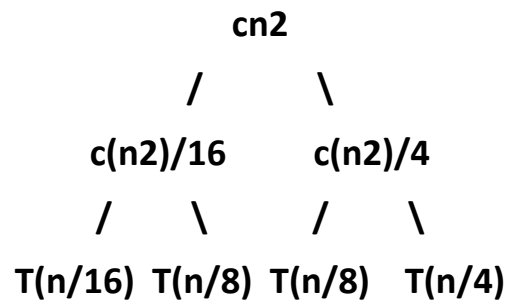
- Represent the recurrence relation as a tree.
- Analyze the total cost by summing up the costs of all levels of the tree.
- In this method, we draw a recurrence tree and calculate the time taken by every level of the tree.
- Finally, we sum the work done at all levels.
- To draw the recurrence tree, we start from the given recurrence and keep drawing till we find a pattern among levels.
- The pattern is typically arithmetic or geometric series.

For example, consider the recurrence relation

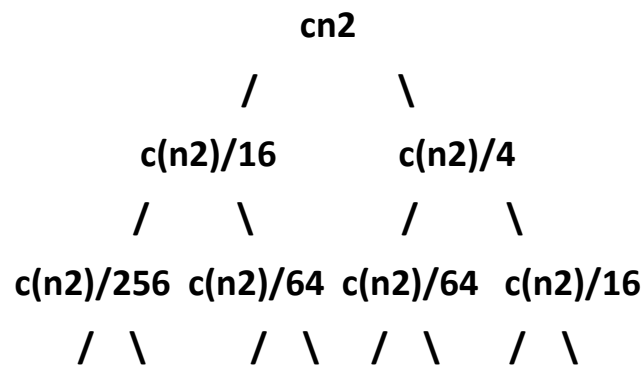
$$T(n) = T(n/4) + T(n/2) + cn^2$$

$$\begin{array}{c} cn^2 \\ / \quad \backslash \\ T(n/4) \quad T(n/2) \end{array}$$

If we further break down the expression **$T(n/4)$** and **$T(n/2)$** , we get the following recursion tree.

Solution:

Breaking down further gives us following



- ✓ To know the value of **T(n)**, we need to calculate the sum of tree
- ✓ nodes level by level.
- ✓ If we sum the above tree level by level,
- ✓ we get the following series **T(n) = c(n² + 5(n²)/16 + 25(n²)/256) +**
- ✓ The above series is a geometrical progression with a ratio of **5/16**.
- ✓ To get an upper bound, we can sum the infinite series.
- ✓ We get the sum as **(n²)/(1 – 5/16)** which is **O(n²)**.

Example: QuickSort Algorithm

Let's apply the recursion tree method to analyze the time and space complexity of the QuickSort algorithm.

Time Complexity:

The partitioning step in QuickSort takes **$O(n)$** time, and the algorithm splits the array into two subarrays of approximately equal size.

The recursion tree for QuickSort has a height of $\log n$ (assuming balanced partitions at each level) = **Number of levels $\times$ Cost per level**

$$T(n) = \text{Number of levels} \times \text{Cost per level}$$

$$= \log T(n) = \log n \times O(n)$$

©Topperworld

$$T(n) = O(n \log n)$$

Space Complexity:

The space complexity of QuickSort is influenced by the recursion stack. At each level, the space used is proportional to the size of the subproblems at that level = **Number of levels $\times$ Space per level**

$$S(n) = \text{Number of levels} \times \text{Space per level}$$

$$\log S(n) = \log n \times O(\log n)$$

$$\log S(n) = O(\log n)$$

The dominant term in the **space complexity** is **$\log n$** , indicating a logarithmic growth in space usage.

So ,

Time Complexity is $O(n)$.

Space Complexity is $\log n$.

❖ Iteration Methods:

- It means to expand the recurrence and express it as a summation of terms of n and initial condition.

Example1:

Consider the Recurrence $T(n) = 1$ if $n=1$
 $= 2T(n-1)$ if $n>1$

Solution:

$$\begin{aligned} T(n) &= 2T(n-1) \\ &= 2[2T(n-2)] = 2^2T(n-2) \\ &= 4[2T(n-3)] = 2^3T(n-3) \\ &= 8[2T(n-4)] = 2^4T(n-4) \end{aligned} \quad \dots(\text{Eq.1})$$

Repeat the procedure for i times

$$T(n) = 2^i T(n-i)$$

Put $n-i=1$ or $i = n-1$ in $\dots(\text{Eq.1})$

$$\begin{aligned} T(n) &= 2^{n-1} T(1) \\ &= 2^{n-1} \cdot 1 \quad \{T(1) = 1 \dots \text{given}\} \\ &= 2^{n-1} \end{aligned}$$

❖ Master Theorem:

- The Master Theorem is a tool used in the analysis of algorithms, particularly those that follow a divide-and-conquer approach.
- It provides a straightforward way to determine the time complexity of recursive algorithms by expressing their time recurrence relations in a specific form.

The general form of a recurrence relation that can be solved using the Master Theorem is: $T(n) = aT(n/b) + f(n)$

Here:

- **T(n)** is the time complexity of the algorithm for a problem of size **n**,
- **a** is the number of subproblems at each recursion level,
- **b** is the factor by which the problem size is reduced in each recursive call,
- **f(n)** is the cost of dividing the problem and combining the results.
- The Master Theorem has three cases, each of which provides a solution for the time complexity **T(n)**

Case1:

If $f(n) = O(n^c)$ where $c < \log_b a$, then the time complexity is:
 $T(n) = \Theta(n^{\log_b a})$

Case2:

If $f(n) = \Theta(n^{\log_b a})$, then the time complexity is:
 $T(n) = \Theta(n^{\log_b a} \log n)$

Case 3:

If $f(n) = \Omega(n^c)$ where $c > \log_b a$, and $af\left(\frac{n}{b}\right) \leq kf(n)$ for some $k < 1$ and

sufficiently large **n**, then the time complexity is:

$$T(n) = \Theta(f(n))$$

The **Master Theorem** provides a convenient way to analyze the time complexity of many **divide-and-conquer** algorithms without going through the process of constructing a recursion tree or using the substitution method. However, it is applicable to a specific form of recurrence relations, and not all recurrence relations fit the Master Theorem's framework. In such cases, other methods like the recursion.